

Playwright (JavaScript)

Complete Interview Guide



Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community ■

Bengaluru, Karnataka, India

[linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)

JSPM University Pune

Core Concepts	Locators	Actions & Assertions
Framework Design	Advanced Topics	API Testing



Table of Contents

CORE CONCEPTS

1. Playwright Architecture
2. Installation & Project Setup
3. Test Runner (test, expect)
4. Browser Context & Page

LOCATORS

1. locator()
2. getByRole() ★
3. getByText()
4. CSS & XPath
5. Best Practices ★

ACTIONS & ASSERTIONS

1. Actions
2. Assertions
3. Auto-Waiting ★
4. Best Practices ★

FRAMEWORK DESIGN

1. Page Object Model (POM) ★
2. Folder Structure
3. Reusability & Utilities
4. Fixtures ★
5. Config Management
6. Data-Driven Testing
7. Reporting
8. CI/CD Integration Basics

ADVANCED TOPICS — PART 1

1. Handling Frames
2. Handling Multiple Tabs / Windows ★

3. File Upload & Download

ADVANCED TOPICS — PART 2

1. Network Interception ★
2. API Mocking ★
3. Debugging Tools
4. Trace Viewer ★
5. Parallel Execution ★
6. Retries & Error Handling

API TESTING WITH PLAYWRIGHT

1. APIRequestContext ★
2. HTTP Methods (GET, POST, PUT, DELETE)
3. Request Headers
4. Authentication (Bearer Token) ★
5. Response Validation
6. Status Code Validation
7. API Chaining ★
8. UI + API Combined Flow ★
9. Best Practices

CORE CONCEPTS

1. Playwright Architecture

What is it

Playwright architecture defines how tests interact with browsers using a Node.js layer + browser engines (Chromium, Firefox, WebKit).

Key Components

- Test Script (JS/TS)
- Playwright API
- Browser Server
- Browser Engines
- Browser Context
- Page

How YOU should answer

Playwright follows a **client-server architecture** where test scripts communicate with browser engines through Playwright APIs. It supports Chromium, Firefox, and WebKit, and uses **browser contexts** for isolated test execution.

Practical Example

```
JavaScript

test('example', async ({ page }) => {
  await page.goto('https://example.com');
});
```

Common Mistakes

- ✗ Saying 'Playwright only works on Chrome'
- ✗ Not mentioning browser context
- ✗ Over-explaining internal workings

Expected Interview Questions

- What is Playwright architecture?
- How is it different from Selenium?
- What is browser context?

2. Installation & Project Setup

What is it

Setting up a Playwright project using Node.js and installing required dependencies.

Key Components

- Node.js
- npm
- @playwright/test
- Browser installation
- Project structure

How YOU should answer

I set up Playwright using Node.js by installing **@playwright/test** via npm. Then I install browser binaries and create a test folder structure for writing test cases.

Practical Example

Bash

```
npm init -y
npm install -D @playwright/test
npx playwright install
```

Common Mistakes

- ✗ Saying 'Playwright needs Selenium'
- ✗ Not knowing setup commands
- ✗ Not knowing project structure

Expected Interview Questions

- How do you install Playwright?
- What are the prerequisites?
- What is the project structure?

3. Test Runner (test, expect)

What is it

Playwright's built-in Test Runner is used to execute tests and perform assertions.

Key Components

- test()
- expect()
- Hooks (beforeEach, afterEach)
- Fixtures

How YOU should answer

Playwright provides a built-in test runner where **test()** defines test cases and **expect()** is used for assertions. It supports fixtures and hooks for better test management.

Practical Example

```
JavaScript

test('title check', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

Common Mistakes

- ✗ Confusing test() with a plain function
- ✗ Not knowing expect() usage
- ✗ Ignoring hooks

Expected Interview Questions

- What is test() in Playwright?
- What is expect()?
- What are hooks?

4. Browser Context & Page

What is it

Browser context is an isolated session, and page represents a tab.

Key Components

- Browser
- Context
- Page

How YOU should answer

Browser context is like a separate session similar to incognito mode, and page represents a single tab. This helps in running tests independently without affecting each other.

Practical Example

```
JavaScript  
  
const context = await browser.newContext();  
const page = await context.newPage();
```

Common Mistakes

- ✗ Saying context = browser
- ✗ Not understanding isolation
- ✗ Confusing page and context

Expected Interview Questions

- What is browser context?
- Difference between browser and context?
- Why is context important?

LOCATORS

1. locator()

What is it

locator() is a Playwright method used to find elements using selectors like CSS, XPath, id, etc.

Key Components

- CSS selectors (#id, .class)
- XPath
- Chaining (locator().locator())
- Auto-waiting support

How YOU should answer

locator() is a flexible method in Playwright used to identify elements using CSS or XPath selectors. It supports auto-waiting and allows chaining for better element targeting.

Practical Example

```
JavaScript
await page.locator('#username').fill('Kalpesh');
await page.locator('.login-btn').click();
```

Common Mistakes

- ✗ Using long XPath unnecessarily
- ✗ Not using chaining
- ✗ Over-relying on CSS when better options exist

Expected Interview Questions

- What is locator() in Playwright?
- Difference between locator() and getByRole()?
- Why is locator better than findElement (Selenium)?

2. getByRole() ★

What is it

Used to locate elements based on ARIA roles (accessibility-based).

Key Components

- Role (button, textbox, link)
- Name (visible label/text)
- Accessibility tree

How YOU should answer

`getByRole()` is the most recommended locator in Playwright as it uses accessibility roles, making tests more stable, readable, and aligned with real user interactions.

Practical Example

```
JavaScript
```

```
await page.getByRole('button', { name: 'Login' }).click();
```

Common Mistakes

- ✗ Not using role-based locators
- ✗ Ignoring accessibility
- ✗ Using XPath instead

Expected Interview Questions

- Why is `getByRole` preferred?
- What is the ARIA role?
- Difference between `getByRole` and `locator`?

3. `getByText()`

What is it

Used to locate elements using visible text on the page.

Key Components

- Exact text match
- Partial text match
- Case sensitivity
- Works on visible UI text

How YOU should answer

`getByText()` is used to locate elements based on visible text. It is simple and readable but less stable compared to role-based locators since UI text can change frequently.

Practical Example

```
JavaScript
await page.getByText('Login').click();
await page.getByText('Submit').click();
```

Common Mistakes

- ✗ Overusing text-based locators
- ✗ Not handling dynamic text
- ✗ Using when role-based locator is better

Expected Interview Questions

- When do you use getByText()?
- What are limitations of text locators?

4. CSS & XPath

What is it

Traditional selector strategies used to locate elements in the DOM.

Key Components

- CSS (#id, .class, tag)
- XPath (//, contains(), text())

How YOU should answer

CSS and XPath are fallback locator strategies. CSS is generally faster and preferred, while XPath is useful for complex DOM traversal when other locators are not sufficient.

Practical Example

```
JavaScript
await page.locator('#username').fill('test'); // CSS
await page.locator('//button[text()="Login"]').click(); // XPath
```

Common Mistakes

- ✗ Writing long and complex XPath
- ✗ Using XPath when better locators exist
- ✗ Not optimising selectors

Expected Interview Questions

- CSS vs XPath?
- Which is faster and why?
- When do you use XPath?

5. Best Practices ★

What is it

Guidelines to write stable, maintainable locators.

Key Components

- Prefer getByRole()
- Use data-testid
- Avoid XPath
- Keep selectors simple
- Use chaining

How YOU should answer

My approach is to prefer role-based locators like getByRole for stability. If not available, I use data-testid or CSS selectors. I avoid XPath unless necessary.

Practical Example

```
JavaScript

await page.getByRole('button', { name: 'Submit' }).click();
await page.locator('[data-testid="login-btn"]').click();
```

Common Mistakes

- ✗ Using absolute XPath
- ✗ Writing brittle selectors
- ✗ Ignoring readability

Expected Interview Questions

- What is your locator strategy?
- Which locator do you prefer and why?
- How do you write stable locators?

ACTIONS & ASSERTIONS

1. Actions

What is it

Actions are methods used to interact with web elements like clicking, typing, selecting, hovering, etc.

Key Components

- click()
- fill()
- type()
- check() / uncheck()
- hover()
- selectOption()
- press()

How YOU should answer

Actions in Playwright simulate real user interactions. Playwright **automatically waits** for elements before performing actions.

Practical Example

```
JavaScript
await page.locator('#username').fill('Kalpesh');
await page.getByRole('button', { name: 'Login' }).click();
await page.locator('#country').selectOption('India');
```

Common Mistakes

- ✗ Using unnecessary waits before actions
- ✗ Using force click without reason
- ✗ Confusing fill() and type()

Expected Interview Questions

- Difference between fill() and type()?
- What actions have you used in Playwright?
- How does Playwright handle waits during actions?

2. Assertions

What is it

Assertions are validations used to verify expected behaviour or UI state.

Key Components

- toHaveText()
- toBeVisible()
- toHaveURL()
- toHaveTitle()
- toBeChecked()

How YOU should answer

Assertions in Playwright validate application behaviour. Playwright provides built-in **auto-retrying assertions** which improve test stability and reduce flaky tests.

Practical Example

JavaScript

```
await expect(page).toHaveTitle(/Dashboard/);  
await expect(page.locator('.success-msg')).toBeVisible();  
await expect(page.locator('#name')).toHaveValue('Kalpesh');
```

Common Mistakes

- ✗ Using manual validations instead of expect()
- ✗ Not understanding auto-retry behaviour
- ✗ Writing weak assertions

Expected Interview Questions

- What is assertion in Playwright?
- Difference between hard and soft assertion?
- Why are Playwright assertions stable?

3. Auto-Waiting ★

What is it

Playwright automatically waits for elements to become ready before performing actions or assertions.

Key Components

- Visibility check
- Element enabled state
- Stability check
- Auto-retrying assertions

How YOU should answer

One major advantage of Playwright is built-in **auto-waiting**. Before performing actions, Playwright ensures elements are visible, stable, and actionable.

Practical Example

```
JavaScript

// No explicit wait required
await page.getByRole('button', { name: 'Submit' }).click();
```

Common Mistakes

- ✗ Adding unnecessary `waitForTimeout()`
- ✗ Using hard waits everywhere
- ✗ Not trusting Playwright auto-wait

Expected Interview Questions

- What is auto-waiting in Playwright?
- Difference between Playwright and Selenium waits?
- Why does Playwright reduce flaky tests?

4. Best Practices ★

What is it

Guidelines for writing reliable actions and assertions.

Key Components

- Prefer built-in assertions
- Avoid hard waits
- Use stable locators
- Validate meaningful UI behaviour

How YOU should answer

I avoid hard waits and rely on Playwright's built-in auto-waiting and assertions. I also use stable locators and meaningful validations.

Practical Example

```
JavaScript
```

```
await expect(page.getByRole('heading')).toHaveText('Dashboard');
```

Common Mistakes

- ✗ Using `waitForTimeout()` frequently
- ✗ Weak validations
- ✗ Over-validating unnecessary UI elements

Expected Interview Questions

- What are best practices for assertions?
- How do you avoid flaky tests?
- Why avoid hard waits?

FRAMEWORK DESIGN

1. Page Object Model (POM) ★

What is it

POM is a design pattern where web pages are represented as separate classes/files containing locators and reusable methods.

Key Components

- Page classes
- Locators
- Reusable methods
- Separation of test logic and page logic

How YOU should answer

POM is a framework design pattern used to improve maintainability and reusability. Locators and page-specific methods are stored separately from test cases.

Practical Example

```
JavaScript

class LoginPage {
  constructor(page) {
    this.page = page;
    this.username = page.locator('#username');
    this.password = page.locator('#password');
  }

  async login(user, pass) {
    await this.username.fill(user);
    await this.password.fill(pass);
  }
}
```

Common Mistakes

- ✗ Writing locators inside test files
- ✗ Mixing test logic and page logic
- ✗ Creating very large page classes

Expected Interview Questions

- What is POM?
- Advantages of POM?
- Why use POM in Playwright?
- Difference between POM and traditional framework?

2. Folder Structure

What is it

Folder structure organises framework components for scalability and maintainability.

Key Components

- pages/
- tests/
- utils/
- fixtures/
- test-data/
- config/

How YOU should answer

A proper folder structure helps maintain scalability and readability. I separate pages, test cases, utilities, fixtures, and test data.

Practical Example

```
Structure

project-root/
  ■■■ pages/
  ■■■ tests/
  ■■■ utils/
  ■■■ fixtures/
  ■■■ test-data/
  ■■■ playwright.config.js
```

Common Mistakes

- ✗ Keeping everything in one file
- ✗ No separation of concerns
- ✗ Improper naming conventions

Expected Interview Questions

- Explain your framework structure
- Why is folder structure important?
- How do you organise test data?

3. Reusability & Utilities

What is it

Utilities are reusable helper methods used across the framework to avoid duplicate code.

Key Components

- Common methods
- Helper functions
- Reusable actions
- Generic validations

How YOU should answer

I create utility methods for reusable operations like login, waits, screenshots, and common validations to reduce code duplication.

Practical Example

```
JavaScript

async function takeScreenshot(page, name) {
  await page.screenshot({ path: `${name}.png` });
}
```

Common Mistakes

- ✗ Duplicate code
- ✗ Large utility files with mixed responsibilities
- ✗ Hardcoded values

Expected Interview Questions

- What utilities have you created?
- How do you improve framework reusability?
- How do you avoid duplicate code?

4. Fixtures ★

What is it

Fixtures are reusable setup components used to share test setup and dependencies across tests.

Key Components

- Shared setup
- Test isolation
- Dependency injection
- beforeEach equivalent behaviour

How YOU should answer

Fixtures in Playwright help manage reusable setup logic and dependencies. They improve test isolation and reduce repeated setup code.

Practical Example

```
JavaScript

test.beforeEach(async ({ page }) => {
  await page.goto('https://example.com');
});
```

Common Mistakes

- ✗ Repeating setup in every test
- ✗ Misusing global setup
- ✗ Not understanding test isolation

Expected Interview Questions

- What are fixtures?
- Why are fixtures useful?
- Difference between hooks and fixtures?

5. Config Management

What is it

Configuration management controls framework settings like base URL, browser setup, retries, and environments.

Key Components

- playwright.config.js
- Base URL
- Retries
- Parallel execution
- Environment setup

How YOU should answer

Playwright configuration helps manage browser settings, execution behaviour, retries, and environment-specific values centrally using **playwright.config.js**.

Practical Example

```
JavaScript
use: {
  baseURL: 'https://example.com',
  headless: true
}
```

Common Mistakes

- ✗ Hardcoding environment values
- ✗ No retry configuration
- ✗ Poor environment management

Expected Interview Questions

- What is playwright.config.js?
- How do you manage multiple environments?
- How do you enable retries?

6. Data-Driven Testing

What is it

Running the same test with multiple datasets.

Key Components

- JSON data
- Parameterised tests
- External test data

How YOU should answer

Data-driven testing helps execute the same test flow with multiple datasets, improving coverage and reducing duplicate test cases.

Practical Example

```
JavaScript
const users = [
  { username: 'admin', password: '1234' },
  { username: 'test', password: '5678' }
];
```

Common Mistakes

- ✗ Hardcoded test data
- ✗ Mixing test data and logic
- ✗ Poor data organisation

Expected Interview Questions

- What is data-driven testing?
- How do you manage test data?
- Why use JSON data?

7. Reporting

What is it

Reporting helps track test execution results and failures.

Key Components

- HTML Reports
- Allure Reports
- Screenshots
- Logs

How YOU should answer

I use Playwright HTML reports and capture screenshots for failed test cases to help analyse execution results efficiently.

Practical Example

```
Bash
```

```
npx playwright show-report
```

Common Mistakes

- ✗ No failure screenshots
- ✗ Ignoring logs
- ✗ Poor report readability

Expected Interview Questions

- What reporting tools have you used?
- How do you debug failed tests?
- Why are reports important?

8. CI/CD Integration Basics

What is it

Integrating automation execution into CI/CD pipelines for continuous testing.

Key Components

- Jenkins
- GitHub Actions
- Automated execution
- Scheduled runs

How YOU should answer

CI/CD integration helps execute automation tests automatically during build or deployment pipelines.

Practical Example

```
Bash
```

```
npx playwright test
```

Common Mistakes

- ✗ Manual-only execution
- ✗ No reporting integration
- ✗ Ignoring pipeline failures

Expected Interview Questions

- How do you integrate Playwright with Jenkins?

- Why is CI/CD important in automation?
- How do you trigger automated execution?

★ Framework Design Priority

1. POM
2. Reusability
3. Fixtures
4. Config Management
5. Reporting
6. CI/CD

ADVANCED TOPICS — PART 1

1. Handling Frames

What is it

Frames (iframes) are embedded HTML documents inside a web page.

Key Components

- iframe
- frameLocator()
- Nested frames
- Frame isolation

How YOU should answer

In Playwright, I use **frameLocator()** to interact with elements inside iframes because normal locators cannot directly access frame elements.

Practical Example

```
JavaScript
await page.frameLocator('#login-frame')
  .locator('#username')
  .fill('Kalpesh');
```

Common Mistakes

- ✗ Trying to access iframe elements directly
- ✗ Using normal locators without frameLocator()
- ✗ Ignoring nested frames

Expected Interview Questions

- What is iframe?
- How do you handle frames in Playwright?
- Difference between page locator and frame locator?

2. Handling Multiple Tabs / Windows ★

What is it

Handling scenarios where clicking an element opens a new browser tab or window.

Key Components

- `context.waitForEvent('page')`
- New page event
- Switching tabs
- Browser context

How YOU should answer

I use `context.waitForEvent('page')` to capture and switch to newly opened tabs.

Practical Example

```
JavaScript

const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('#open-tab')
]);

await newPage.waitForLoadState();
```

Common Mistakes

- ✗ Not waiting for new page event
- ✗ Confusing browser context and page
- ✗ Hardcoding tab switching logic

Expected Interview Questions

- How do you handle multiple tabs in Playwright?
- Difference between context and page?
- How do you switch between tabs?

3. File Upload & Download

What is it

Handling file upload and file download scenarios in automation testing.

Key Components

- `setInputFiles()`
- Download event
- File path handling

- Upload validation

How YOU should answer

Playwright provides built-in support for file upload and download. I use **setInputFiles()** for uploads and download event listeners for validating downloaded files.

File Upload Example

```
JavaScript

await page.locator('input[type="file"]')
.setInputFiles('test-data/sample.pdf');
```

File Download Example

```
JavaScript

const downloadPromise = page.waitForEvent('download');
await page.click('#download-btn');
const download = await downloadPromise;
```

Common Mistakes

- ✗ Using OS-level automation unnecessarily
- ✗ Hardcoding file paths
- ✗ Not validating downloaded file

Expected Interview Questions

- How do you upload files in Playwright?
- How do you validate downloads?
- Difference between Selenium and Playwright file handling?

Concept	Key Method
iframe handling	frameLocator()
New tab handling	context.waitForEvent('page')
File upload	setInputFiles()

ADVANCED TOPICS — PART 2

1. Network Interception ★

What is it

Network interception allows capturing, monitoring, modifying, or blocking API/network requests during test execution.

Key Components

- `page.route()`
- Request interception
- Response modification
- Blocking requests

How YOU should answer

Network interception allows us to monitor and modify network requests and responses. Useful for validating APIs, mocking responses, and handling unstable backend dependencies.

Practical Example

```
JavaScript

await page.route('**/api/users', async route => {
  await route.continue();
});
```

Common Mistakes

- ✗ Blocking all requests unnecessarily
- ✗ Incorrect URL patterns
- ✗ Not understanding request flow

Expected Interview Questions

- What is network interception?
- Why use `route()` in Playwright?
- Real-time use case of interception?

2. API Mocking ★

What is it

API mocking means simulating backend responses without hitting the actual server.

Key Components

- Mock responses
- `route.fulfill()`
- Fake API data
- Isolation testing

How YOU should answer

API mocking helps test frontend behaviour independently from backend systems. I use **`route.fulfill()`** to return custom mock responses.

Practical Example

```
JavaScript

await page.route('**/api/login', async route => {
  await route.fulfill({
    status: 200,
    body: JSON.stringify({ success: true })
  });
});
```

Common Mistakes

- ✗ Mocking unnecessary APIs
- ✗ Returning invalid response structure
- ✗ Overusing mocks in all tests

Expected Interview Questions

- What is API mocking?
- Difference between mocking and interception?
- Why use mocked APIs?

3. Debugging Tools

What is it

Playwright provides debugging tools to identify failures and analyse test behaviour.

Key Components

- `page.pause()`
- `PWDEBUG`
- Console logs
- Debug mode

How YOU should answer

Playwright debugging tools help analyse failures interactively. I use `page.pause()`, debug mode, and logs to inspect application state.

Practical Example

```
Bash
# Add page.pause() in test, then run:
PWDEBUG=1 npx playwright test
```

Common Mistakes

- ✗ Leaving `pause()` in production code
- ✗ Ignoring logs
- ✗ Not using debugging tools properly

Expected Interview Questions

- How do you debug Playwright tests?
- What is `PWDEBUG`?
- How do you analyse failures?

4. Trace Viewer ★

What is it

Trace Viewer is a Playwright tool used to visually analyse test execution step-by-step.

Key Components

- Screenshots
- DOM snapshots
- Network logs
- Execution timeline

How YOU should answer

Trace Viewer helps analyse failed test executions visually by providing screenshots, DOM snapshots, and network activity for each step.

Enable Tracing (playwright.config.js)

```
JavaScript
use: {
  trace: "on-first-retry"
}
```

Open Trace

```
Bash
npx playwright show-trace trace.zip
```

Common Mistakes

- ✗ Enabling tracing for all executions unnecessarily
- ✗ Not analysing traces properly
- ✗ Ignoring network logs

Expected Interview Questions

- What is Trace Viewer?
- How do you analyse failed tests?
- Difference between report and trace?

5. Parallel Execution ★

What is it

Running multiple tests simultaneously to reduce execution time.

Key Components

- Workers
- Parallel test execution
- Resource optimisation
- Independent tests

How YOU should answer

Parallel execution allows multiple tests to run simultaneously using Playwright workers, improving execution speed and CI/CD efficiency.

Practical Example

```
JavaScript
```

```
// playwright.config.js  
workers: 4
```

Common Mistakes

- ✗ Shared test data issues
- ✗ Dependent test cases
- ✗ Environment conflicts

Expected Interview Questions

- What is parallel execution?
- How does Playwright run tests in parallel?
- Challenges in parallel execution?

6. Retries & Error Handling

What is it

Retries help rerun failed tests automatically, while error handling manages failures gracefully.

Key Components

- retries
- try-catch
- Failure handling
- Flaky test management

How YOU should answer

Retries in Playwright help reduce flaky failures by rerunning failed tests automatically. I also use proper error handling and logging for better debugging.

Practical Example

```
JavaScript  
// playwright.config.js  
retries: 2
```

Common Mistakes

- ✗ Overusing retries to hide real issues
- ✗ No proper logging
- ✗ Ignoring flaky tests

Expected Interview Questions

- What are retries in Playwright?
- How do you handle flaky tests?
- Why should retries be used carefully?

Feature	Command
Debug Mode	PWDEBUG=1 npx playwright test
Trace Viewer	npx playwright show-trace trace.zip
Parallel Workers	workers: 4 in config

API TESTING WITH PLAYWRIGHT

1. APIRequestContext ★

What is it

APIRequestContext is Playwright's built-in API testing utility used to send HTTP requests directly without browser interaction.

Key Components

- request
- API context
- HTTP requests
- Base URL

How YOU should answer

APIRequestContext is used for backend/API testing without launching the browser. It helps perform direct HTTP operations like GET, POST, PUT, and DELETE.

Practical Example

```
JavaScript
const response = await request.get('/users');
```

Common Mistakes

- ✗ Confusing browser page and API request
- ✗ Hardcoding endpoints
- ✗ Not validating responses

Expected Interview Questions

- What is APIRequestContext?
- Why use API testing in Playwright?
- Difference between UI and API testing?

2. HTTP Methods (GET, POST, PUT, DELETE)

What is it

HTTP methods are operations used to interact with APIs.

Key Components

- GET — Fetch data
- POST — Create data
- PUT — Update data
- DELETE — Remove data

How YOU should answer

GET retrieves data, POST creates data, PUT updates resources, and DELETE removes resources.

Practical Example

```
JavaScript

await request.get('/users');

await request.post('/users', {
  data: { name: 'Kalpesh' }
});
```

Common Mistakes

- ✗ Using wrong HTTP methods
- ✗ Not validating response codes
- ✗ Sending incorrect payloads

Expected Interview Questions

- Difference between PUT and POST?
- Which HTTP methods have you used?
- What is idempotency?

3. Request Headers

What is it

Headers provide additional information in API requests such as authentication and content type.

Key Components

- Authorization
- Content-Type
- Accept
- Custom headers

How YOU should answer

Request headers are used to pass metadata such as authentication tokens and content type information during API communication.

Practical Example

```
JavaScript
await request.get('/users', {
  headers: { Authorization: 'Bearer token' }
});
```

Common Mistakes

- ✗ Missing authentication headers
- ✗ Incorrect content types
- ✗ Hardcoding sensitive tokens

Expected Interview Questions

- What are request headers?
- Why use Authorization header?
- Difference between headers and body?

4. Authentication (Bearer Token) ★

What is it

Authentication verifies user identity before accessing protected APIs.

Key Components

- Bearer token
- JWT token
- Authorization header
- Session handling

How YOU should answer

Bearer token authentication is commonly used in APIs where a token is passed in the Authorization header to access secured endpoints.

Practical Example

```
JavaScript
```

```
headers: {  
  Authorization: `Bearer ${token}`  
}
```

Common Mistakes

- ✗ Exposing tokens in code
- ✗ Expired tokens
- ✗ Incorrect token format

Expected Interview Questions

- What is bearer token authentication?
- How do you handle tokens securely?
- Difference between session and token auth?

5. Response Validation

What is it

Validating API response body and returned data.

Key Components

- JSON validation
- Response body
- Field validation
- Schema checks

How YOU should answer

Response validation ensures the API returns correct and expected data. I validate fields, response structure, and important business values.

Practical Example

```
JavaScript  
  
const body = await response.json();  
expect(body.name).toBe('Kalpesh');
```

Common Mistakes

- ✗ Validating only status codes
- ✗ Ignoring response body
- ✗ Weak assertions

Expected Interview Questions

- How do you validate API responses?
- What validations do you perform?
- Difference between schema and field validation?

6. Status Code Validation

What is it

Validating API response status codes.

Key Components

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 500 Server Error

How YOU should answer

Status code validation confirms whether API operations are successful or failed as expected.

Practical Example

```
JavaScript
expect(response.status()).toBe(200);
```

Common Mistakes

- ✗ Ignoring failure codes
- ✗ Hardcoding wrong expectations
- ✗ Not validating negative scenarios

Expected Interview Questions

- Common HTTP status codes?
- Difference between 401 and 403?
- Why validate status codes?

7. API Chaining ★

What is it

Using data from one API response in another API request.

Key Components

- Dynamic IDs
- Sequential requests
- Dependency handling

How YOU should answer

API chaining is used when one API response provides data required for another API request, such as user IDs or authentication tokens.

Practical Example

```
JavaScript

const createUser = await request.post('/users');
const body = await createUser.json();
await request.get(`/users/${body.id}`);
```

Common Mistakes

- ✗ Hardcoding IDs
- ✗ Ignoring dependency failures
- ✗ Poor response handling

Expected Interview Questions

- What is API chaining?
- Real-time example of chaining?
- Why is chaining important?

8. UI + API Combined Flow ★

What is it

Combining API and UI testing together for faster and more efficient automation.

Key Components

- Backend setup
- UI validation
- Faster execution
- Reduced UI dependency

How YOU should answer

In real-time projects, I use APIs to create or prepare test data and then validate functionality through UI. This improves execution speed and reduces UI dependency.

Practical Example

```
JavaScript
await request.post('/users', { data: { name: 'Kalpesh' } });
await page.reload();
await expect(page.getByText('Kalpesh')).toBeVisible();
```

Common Mistakes

- ✗ Creating all data through UI
- ✗ Ignoring backend validation
- ✗ Slow test setup

Expected Interview Questions

- Why combine API and UI testing?
- Real-time use case?
- Benefits of API-driven setup?

9. Best Practices ★

What is it

Guidelines for writing stable and maintainable API tests.

Key Components

- Reusable request methods
- Proper validations
- Secure token handling
- Avoid hardcoded data

How YOU should answer

I focus on reusable API utilities, proper response validations, secure token management, and meaningful assertions to maintain reliable API automation.

Practical Example

```
JavaScript
```

```
expect(response.ok()).toBeTruthy();
```

Common Mistakes

- ✗ Hardcoded test data
- ✗ No negative testing
- ✗ Weak assertions

Expected Interview Questions

- API automation best practices?
- How do you secure API tests?
- How do you manage reusable APIs?

★ Most Important API Topics

1. API Chaining
2. Authentication (Bearer Token)
3. Response Validation
4. UI + API Combined Flow

Validation	Purpose
Status Code	Operation success
Response Body	Data correctness
Headers	Metadata validation
Schema	Structure validation

Kalpesh Jain | SDET @ Algebrik AI | [linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)